

Chapter 1

Processes

A *program* is a file. A *process* is a program being executed: an address space with code and data in it, at least one thread of execution, and a set of resources the kernel keeps on its behalf (open files, the current directory, a process id). This chapter looks at processes from the outside — through the system calls that create, replace, wait for and signal them. Team F implements `fork` and `exec` in SWEB for A1; Team P needs the same picture, because a thread lives *inside* a process.

1.1 What the kernel keeps per process

For every process the kernel has a record — textbooks call it the *process control block* (PCB):

- identity: process id (PID), parent PID, user id;
- the address space: page tables and the list of mapped regions;
- open files: the file-descriptor table;
- the current working directory, signal handlers, exit status;
- one or more threads, each with its own saved registers, stack and scheduling state (chapter 2).

The last point is exactly what SWEB does not have yet: there, a process *is* a thread (`class UserProcess : public Thread`), and the first thing your group does in A1 (“Phase 0”) is to split the two.

1.2 The address space

Every process sees its own, private range of virtual addresses (chapter 8 explains how the hardware makes that possible). On x86-64 Linux the lower half belongs to the process, the upper half to the kernel (figure 1.1).

The heap grows upwards (`malloc` asks the kernel for more with `brk/mmap`), the stack downwards. `/proc/<pid>/maps` shows the real layout of a running process (module 08’s `demand_paging` example prints it).

In SWEB

The boundary is `USER_BREAK` (`arch/x86/64/include/offsets.h`): `0x0000800000000000`. A new SWEB process gets exactly one stack page, the last page below `USER_BREAK`, and its stack pointer starts at `USER_BREAK - 8`. Code and data are *not* loaded up front: each first access page-faults and `Loader::loadPage` copies the page from the binary (chapter 8).

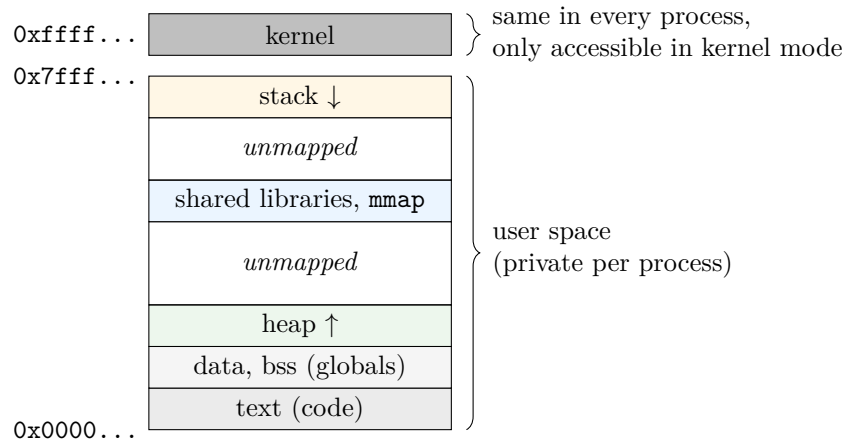


Figure 1.1: The address space of a process on x86-64 Linux (not to scale).

1.3 fork, exec, wait, exit

fork() creates a copy of the calling process: same code, a copy of all memory, copies of the file descriptors (pointing to the same open files). It is called once and *returns twice* — in the parent with the child’s PID, in the child with 0:

```
pid_t pid = fork();
if (pid < 0) { perror("fork"); } /* no child */
else if (pid == 0) { /* child */ }
else { /* parent: pid = the child's PID */ }
```

“Returns twice” is less magic than it sounds: the kernel copies the parent’s saved user registers into the child and then changes one of them — the return-value register (**rax** on x86-64) is 0 in the child. When each of them is next scheduled, both continue at the instruction after the system call. Copying all memory would be slow; real kernels share the pages and copy a page only when one side writes to it (*copy-on-write*, chapter 8).

exec*() replaces the program running in the current process: new code, new data, new stack, same PID, same open file descriptors (unless marked close-on-exec). On success it does not return — the code that called it no longer exists.

exit(status) and wait. A process ends with an exit status; its parent collects it with `wait/waitpid`:

```
int status;
waitpid(pid, &status, 0);
if (WIFEXITED(status)) printf("exit code %d\n", WEXITSTATUS(status));
else if (WIFSIGNALED(status)) printf("killed by signal %d\n", WTERMSIG(status));
```

The shell does exactly this for every command: `fork`, `exec` in the child, `waitpid` in the parent (module 01’s `minish`).

Zombies and orphans. A process that has exited but whose status has not been collected yet is a *zombie*: its memory is gone, but its PCB with the exit status must stay until the parent calls `wait`. A process whose parent dies first is an *orphan*; it is adopted by `init` (PID 1), which reaps it. The same idea returns for threads: a finished, not yet joined thread keeps its return value (chapter 10).

1.4 File descriptors and pipes

A file descriptor is an index into the process's descriptor table; each entry points to an *open file* object in the kernel (position, flags, the file or pipe itself). By convention 0, 1 and 2 are standard input, output and error.

- `fork` copies the table: parent and child share the same open files (and the same file position).
- `dup2(old, new)` makes `new` refer to the same open file as `old` — this is how a shell redirects output.
- `pipe(fds)` creates a kernel buffer with a read end (`fds[0]`) and a write end (`fds[1]`). A reader blocks while the buffer is empty, a writer while it is full.

A reader sees end-of-file only when *every* write end is closed — in every process. A forgotten write end in the parent keeps `wc` waiting forever. Rule: after `fork + dup2`, close every pipe end you do not use, in every process.

1.5 Signals

A signal is an asynchronous notification to a process: `SIGINT` (Ctrl-C), `SIGSEGV` (bad memory access), `SIGCHLD` (a child changed state), `SIGALRM` (a timer expired), `SIGKILL` (cannot be caught). Each has a default action (terminate, core dump, ignore, stop); `sigaction` installs a handler instead.

A handler interrupts the program at an arbitrary instruction — like a hardware interrupt interrupts the kernel. Consequently a handler may only call *async-signal-safe* functions (`write`, `_exit`, ..., but not `printf` or `malloc`, which may hold internal locks at the moment of interruption) and should communicate through a `volatile sig_atomic_t` flag. Chapter 10 uses a timer signal to preempt user-level threads and runs into exactly this rule.

1.6 A trap: buffered output and fork

`printf` does not write immediately: `stdio` collects output in a buffer inside the process and flushes it at a newline (terminal), when the buffer is full (pipe, file) or at exit. `fork` copies that buffer too — output that was “printed” before `fork` but not yet flushed appears *twice*, once from each process. Call `fflush(stdout)` before `fork` (module 01, `stdio_trap.c`; module 08's `cow_fork.c` fell into it while being written).

In SWEB

Baseline SWEB has no `fork` and no `exec`: the shell starts programs with the course's own `createprocess` system call (`common/source/kernel/Syscall.cpp`), which creates a new `UserProcess` for a binary. Implementing `fork` with copy-on-write and `exec` is Team F's part of A1; tearing down every thread of a process on `exit` is the joint work of both teams.

Key points

- A process = address space + resources + threads; the kernel keeps a record (PCB) for each.
- **fork** returns twice because the child gets a copy of the parent's registers with the return value set to 0.
- **exec** replaces the program, keeps the PID and the open files.
- An exited process stays a zombie until its parent waits for it.
- Close unused pipe ends; flush **stdio** before **fork**.
- Signal handlers run at arbitrary points: only async-signal-safe calls inside.